
PropCRM

Lead Management System for Real Estate Agents

DATA FLOW ARCHITECTURE DOCUMENT

Document Version	1.0
Date	July 7, 2026
Status	Draft — Pending Approval
Author	Product Engineering Team
Related Document	PRD v1.0

CONFIDENTIAL — INTERNAL USE ONLY

1. System Architecture Overview

PropCRM follows a **three-tier architecture** with clear separation between the presentation layer (GitHub Pages), application layer (Google Apps Script), and data layer (Google Sheets). All communication between tiers occurs over HTTPS using JSON payloads.

1.1 Architecture Layers

LAYER	COMPONENT	TECHNOLOGY	RESPONSIBILITY
Presentation	Lead Form	HTML/CSS/JS (GitHub Pages)	Lead data capture, client-side validation, UX feedback
Presentation	Agent Dashboard	HTML/CSS/JS + Chart.js (GitHub Pages)	Lead management, analytics visualization, email actions
Application	API Gateway	Google Apps Script (Web App)	Request routing, server-side validation, business logic, email orchestration
Data	Lead Store	Google Sheets — "Leads" sheet	Persistent storage of all lead records
Data	Config Store	Google Sheets — "AgentConfig" sheet	Agent settings, API keys, authentication PIN
External	Email Service	Brevo v3 REST API	Transactional email delivery

1.2 Communication Protocols

PATH	PROTOCOL	FORMAT	AUTH
Frontend → Apps Script	HTTPS (fetch API)	JSON	PIN in request body (dashboard only)

PATH	PROTOCOL	FORMAT	AUTH
Apps Script → Google Sheets	SpreadsheetApp (native)	Internal	Script owner's Google account
Apps Script → Brevo	HTTPS (UrlFetchApp)	JSON	API key in header

2. Data Entities & Schema

Entity: Lead

Represents a single property inquiry submitted by a customer.

FIELD	TYPE	SOURCE	CONSTRAINTS	DESCRIPTION
id	String	System-generated	Unique, non-null	UUID v4 identifier
name	String	User input	Min 2 chars, alphabetic + spaces	Full name of the lead
email	String	User input	Valid email format (RFC 5322)	Contact email
area	String	User input	Non-empty, max 100 chars	Property locality/area name
price	Number	User input	Positive integer, > 0	Property budget in ₹
priceBracket	String (Enum)	Computed	One of 6 predefined brackets	Auto-classified price range
submittedAt	DateTime	System-generated	ISO 8601 format	Timestamp of form submission
contacted	Boolean	Agent action	Default: FALSE	Whether agent has followed up
contactedAt	DateTime	System-generated	Nullable	When contacted was set to TRUE

FIELD	TYPE	SOURCE	CONSTRAINTS	DESCRIPTION
<code>emailSentToLead</code>	Boolean	System-generated	Default: FALSE	Acknowledgment email status
<code>reminderSentToAgent</code>	Boolean	System-generated	Default: FALSE	Whether reminder was sent for this lead

Entity: AgentConfig (Singleton)

System configuration for the agent. Single row in the "AgentConfig" sheet.

FIELD	TYPE	DESCRIPTION
<code>agentName</code>	String	Agent's display name (used in email templates)
<code>agentEmail</code>	String	Agent's email for receiving notifications
<code>brevoApiKey</code>	String	Brevo API key for sending emails
<code>dashboardPin</code>	String	PIN for dashboard authentication
<code>reminderHours</code>	Number	Hours before reminder trigger (default: 1)

3. API Endpoints

3.1 Base URL

```
https://script.google.com/macros/s/{DEPLOYMENT_ID}/exec
```

3.2 POST — Submit Lead

POST /exec

FIELD	DETAIL
Action	"submitLead"
Request Body	{ "action": "submitLead", "name": "Rahul Sharma", "email": "rahul@example.com", "area": "Whitefield", "price": 7500000 }
Success Response	{ "success": true, "id": "a1b2c3d4-...", "message": "Lead saved successfully" }
Error Response	{ "success": false, "errors": ["Invalid email format", "Price must be a positive number"] }
Side Effects	1) Row written to Leads sheet 2) Acknowledgment email sent to lead 3) Notification email sent to agent

3.3 POST — Verify PIN

POST /exec

FIELD	DETAIL
Action	"verifyPin"
Request Body	{ "action": "verifyPin", "pin": "1234" }

FIELD	DETAIL
Success	<code>{ "success": true, "agentName": "Rahul" }</code>
Failure	<code>{ "success": false, "error": "Invalid PIN" }</code>

3.4 POST — Update Contacted Status

POST /exec

FIELD	DETAIL
Action	<code>"updateContacted"</code>
Request Body	<code>{ "action": "updateContacted", "pin": "1234", "id": "a1b2c3d4-...", "contacted": true }</code>
Success	<code>{ "success": true }</code>
Side Effects	Updates <code>contacted</code> to TRUE and sets <code>contactedAt</code> to current timestamp

3.5 POST — Send Email to Lead

POST /exec

FIELD	DETAIL
Action	<code>"sendEmailToLead"</code>
Request Body	<code>{ "action": "sendEmailToLead", "pin": "1234", "id": "a1b2c3d4-..." }</code>
Success	<code>{ "success": true, "message": "Email sent" }</code>
Side Effects	Follow-up email sent to lead via Brevo

3.6 GET — Fetch All Leads & Stats

GET /exec?action=getLeads&pin=1234

FIELD	DETAIL
Query Params	<code>action=getLeads</code> , <code>pin=XXXX</code>
Success Response	JSON object containing: <code>leads[]</code> (all lead records), <code>stats</code> (computed metrics), <code>priceBrackets</code> (chart data), <code>areaCounts</code> (chart data)
Auth Failure	<code>{ "success": false, "error": "Invalid PIN" }</code>

4. Data Flow Scenarios

4.1 Flow 1: Lead Submission (Happy Path)

Trigger: Customer clicks "Submit" on the lead form

1

Client-Side Validation

Browser validates all fields (name length, email format, area non-empty, price > 0). If invalid → show inline errors, halt submission.

2

HTTP POST to Apps Script

Form data sent as JSON via `fetch()` to the Apps Script Web App URL. Submit button shows loading spinner.

3

Server-Side Validation

Apps Script re-validates all fields server-side. If invalid → returns `{ success: false, errors: [...] } .`

4

Generate Lead Record

System generates UUID, calculates price bracket, captures timestamp. Constructs complete lead record.

5

Write to Google Sheets

Appends new row to the "Leads" sheet with all 11 columns. Default values:

```
contacted = FALSE , emailSentToLead = FALSE , reminderSentToAgent = FALSE .
```

6

Send Acknowledgment Email to Lead

Apps Script reads Brevo API key from AgentConfig, sends acknowledgment email to the lead's email via Brevo v3 API. Updates `emailSentToLead = TRUE .`

7

Send Notification Email to Agent

Apps Script sends a notification email to the agent's email (from AgentConfig) with the lead's details.

8

Return Success Response

Apps Script returns `{ success: true, id: "..."} . Frontend shows success toast, resets form.`

4.2 Flow 2: Dashboard Data Load

Trigger: Agent opens dashboard and enters correct PIN

1

PIN Verification

Agent enters PIN → POST to Apps Script with `action: "verifyPin"` . On success, PIN is stored in session memory (JavaScript variable). On failure, error message shown.

2

Fetch Leads Data

GET request to Apps Script with `action=getLeads&pin=XXXX` . Apps Script verifies PIN, reads all rows from "Leads" sheet.

3

Compute Statistics (Server-Side)

Apps Script computes: total leads, contacted count, uncontacted count, contacted %, average price, price bracket distribution (count per bracket), area distribution (count per locality).

4

Return Aggregated Response

Returns JSON with `leads[] , stats{} , priceBrackets{} , areaCounts{} .`

5

Render Dashboard

Frontend populates: stat cards, Chart.js doughnut chart (price brackets), Chart.js bar chart (area distribution), leads data table. Green gradient colors applied based on data concentration.

4.3 Flow 3: Mark Lead as Contacted

Trigger: Agent checks the "Contacted" checkbox on a lead row

1

Checkbox Toggle Event

JavaScript captures change event on the checkbox for the specific lead row.

2

POST Update Request

Sends `{ action: "updateContacted", id: "...", contacted: true, pin: "...", }` to Apps Script.

3

Update Google Sheet

Apps Script finds the row by ID, sets `contacted = TRUE` and `contactedAt = NOW()` .

4

UI Confirmation

Frontend updates the row visually (e.g., reduced opacity or green highlight) and updates stat cards.

4.4 Flow 4: Send Follow-up Email from Dashboard

Trigger: Agent clicks "Send Email" button on a lead row

1

Button Click Event

Agent clicks "Send Email" → button changes to loading state.

2

POST Email Request

Sends `{ action: "sendEmailToLead", id: "...", pin: "...", }` to Apps Script.

3

Apps Script Processes Email

Reads lead data by ID from Sheets, reads agent config (name, Brevo key), constructs follow-up email HTML, sends via Brevo API.

4**UI Confirmation**

Button shows ✓ checkmark with "Sent" text. Reverts to normal after 3 seconds.

4.5 Flow 5: Automated Reminder for Uncontacted Leads

Trigger: Time-driven trigger (runs every 1 hour automatically in Apps Script)

1**Scheduled Trigger Fires**

Google Apps Script time-driven trigger executes `checkUncontactedLeads()` every 1 hour.

2**Query Uncontacted Leads**

Reads "Leads" sheet. Filters for rows where: `contacted = FALSE AND reminderSentToAgent = FALSE AND submittedAt` is more than `reminderHours` ago.

3**Send Reminder Emails**

For each qualifying lead, sends a reminder email to the agent via Brevo with lead details and urgency message.

4**Update Reminder Flag**

Sets `reminderSentToAgent = TRUE` for each processed lead to prevent duplicate reminders.

5. Data Security & Privacy

5.1 Sensitive Data Handling

DATA ITEM	CLASSIFICATION	STORAGE	PROTECTION
Brevo API Key	Secret	AgentConfig sheet (cell C1)	Never sent to frontend; accessed only by Apps Script server-side
Dashboard PIN	Secret	AgentConfig sheet (cell D1)	Verified server-side; not stored in browser localStorage
Lead Email	PII	Leads sheet	Accessible only via authenticated dashboard (PIN required)
Lead Name	PII	Leads sheet	Same as above

5.2 Security Controls

CONTROL	IMPLEMENTATION
Transport encryption	All communication over HTTPS (enforced by GitHub Pages and Apps Script)
API key protection	Brevo API key stored in Sheets, accessed only via Apps Script backend
Input sanitization	HTML entities escaped, script tags stripped before writing to Sheets
Authentication	PIN verified on every dashboard API call (passed in request body/query)
Rate limiting	Inherent via Google Apps Script execution quotas (30 concurrent)

⚠ Security Limitation (v1): PIN-based authentication is suitable for single-agent use but is not enterprise-grade. The PIN is sent in request bodies over HTTPS, which is encrypted in transit but could be observed in browser developer tools. For multi-agent or sensitive deployments, upgrading to Google OAuth is recommended in v2.

6. Error Handling Strategy

ERROR SCENARIO	DETECTION	RESPONSE	USER IMPACT
Client-side validation failure	Before API call	Inline error messages on form fields	Form not submitted; user corrects errors
Server-side validation failure	Apps Script validation	Returns <code>{ success: false, errors: [...] }</code>	Error toast shown; form data preserved
Google Sheets write failure	Try-catch in Apps Script	Returns <code>{ success: false, error: "Storage error" }</code>	Error toast; user asked to retry
Brevo API failure (email not sent)	HTTP response status check	Lead still saved; email flag stays FALSE; logged for retry	Lead captured (no data loss); email delivery delayed
Network failure (frontend → Apps Script)	Fetch catch block	Error toast: "Network error. Please check your connection."	Form data preserved; user retries
Invalid PIN on dashboard	PIN verification endpoint	Returns <code>{ success: false }</code>	Error message; login form remains visible
Apps Script quota exceeded	Google error response	HTTP 429 or error message	Temporary unavailability; retry after cooldown

Design Principle: Email failures are non-blocking. The lead is always saved first. Email sending is a best-effort secondary action. If Brevo fails, the lead data is still safely stored in Google Sheets and visible on the dashboard.

7. Data Transformations

7.1 Price Bracket Calculation

Performed server-side in Apps Script when a new lead is submitted:

INPUT PRICE (₹)	BRACKET OUTPUT
0 – 24,99,999	"Under 25L"
25,00,000 – 49,99,999	"25L – 50L"
50,00,000 – 99,99,999	"50L – 1Cr"
1,00,00,000 – 1,99,99,999	"1Cr – 2Cr"
2,00,00,000 – 4,99,99,999	"2Cr – 5Cr"
≥ 5,00,00,000	"5Cr+ "

7.2 Statistics Aggregation

Computed in Apps Script on every dashboard data fetch (GET request):

STAT	COMPUTATION	OUTPUT FORMAT
Total Leads	COUNT(all rows)	Integer
Contacted Count	COUNT(WHERE contacted = TRUE)	Integer
Uncontacted Count	Total - Contacted	Integer
Contacted %	(Contacted / Total) × 100	Percentage (1 decimal)
Average Price	SUM(price) / COUNT	₹ formatted
Price Distribution	GROUP BY priceBracket → COUNT	Object: { bracket: count }
Area Distribution	GROUP BY area (case-insensitive) → COUNT	Object: { area: count }, sorted desc

7.3 Chart Color Mapping (Green Gradient)

Applied on the frontend when rendering Chart.js charts:

CONCENTRATION LEVEL	COLOR	HEX
Lowest (bottom 16%)	Pale Green	#E8F5E9
Low (16–33%)	Light Green	#C8E6C9
Medium-Low (33–50%)	Medium Light Green	#A5D6A7
Medium-High (50–66%)	Medium Green	#81C784
High (66–83%)	Dark Green	#4CAF50
Highest (top 16%)	Deep Green	#2E7D32